



A

Cahier des Charges

Plan Technique IA

Module premium pour AVRA

Version 1.0 — 24 avril 2026

Destiné à Claude Code / équipe dev senior

Document confidentiel — Ne pas diffuser hors AVRA

Auteur : Esteve Boucheret (lumeasolutions@outlook.fr)

Sommaire

1. Contexte & Vision produit	p. 3
2. Glossaire métier complet	p. 7
3. User stories détaillées (Gherkin)	p. 10
4. Architecture technique globale	p. 15
5. Schéma Prisma complet	p. 19
6. API Endpoints (contract)	p. 23
7. Pipeline IA détaillé	p. 27
8. Moteur de règles métier	p. 33
9. Génération PDF (react-pdf + SVG)	p. 39
10. UI / UX frontend	p. 43
11. Stratégie de tests (Gold Standard)	p. 47
12. Observabilité & Ops production	p. 51
13. Sécurité, audit trail & conformité	p. 54
14. Plan de livraison 4-6 mois	p. 57
15. Pricing & business model	p. 60
16. Risques & mitigation	p. 62
17. Annexes (prompts, schémas, fixtures)	p. 64

1. Contexte & Vision produit

1.1 Qu'est-ce qu'AVRA ?

AVRA est un SaaS ERP B2B français conçu pour les **cuisinistes, menuisiers-agenceurs et architectes d'intérieur**. L'application est actuellement en bêta privée (users whitelist) avec un lancement officiel prévu en **juillet 2026**. Elle couvre aujourd'hui : gestion des dossiers clients, suivi des étapes de vente (lead → devis → signature → chantier → réception), facturation e-invoice conforme 2026, planning, stock, intervenants, signature électronique, et un premier module IA de rendu photoréaliste.

Stack technique : **Next.js 14 App Router** (frontend), **NestJS 10** (backend), **Prisma 5.22 + Supabase PostgreSQL** (DB), **Supabase Storage** (fichiers privés, URL signées), déploiement **Vercel** (frontend + serverless NestJS via apps/web/api/index.ts). Monorepo pnpm workspaces. Auth JWT access (15 min) + refresh (30j) en cookies httpOnly SameSite=Lax.

1.2 Le problème métier à résoudre

Un cuisiniste ou menuisier-agenceur passe en moyenne **4 à 8 heures par projet** à produire son **Plan Technique DCE** (Dossier de Consultation des Entreprises). Ce document interne est destiné à l'atelier de fabrication, aux fournisseurs de quincaillerie (commandes précises) et au poseur (fiche de pose).

Aujourd'hui, la production d'un plan technique nécessite l'utilisation de logiciels de conception professionnels (Polyboard, WinnerFlex, KD Max, Imos, SketchUp) qui coûtent entre **4 000 et 15 000 €/licence**, demandent **6 mois de formation**, et ne sont jamais à jour avec les catalogues fournisseurs réels. Résultat :

- Un cuisiniste indépendant passe 15-20h/semaine sur ces plans = 60-80h/mois perdues
- Valorisation : 45€/h × 70h = **~3 150 €/mois de temps non-facturable**
- Les grands groupes (Schmidt, Mobalpa) ont des bureaux d'études dédiés et écrasent les indépendants sur les délais (6 semaines vs 10 semaines)
- Erreurs manuelles fréquentes (une cote fausse = un meuble à refaire = 500-5 000 € de perte)

1.3 La proposition de valeur AVRA

Le module **Plan Technique IA** permet à un artisan, depuis son dossier client AVRA existant, de cliquer sur un bouton « *Générer le Plan Technique* » et d'obtenir en quelques minutes un PDF technique complet de 10-15 pages, prêt à envoyer à l'atelier :

- **Vue éclatée par caisson** : chaque meuble décomposé pièce par pièce
- **Nomenclature chiffrée** : dimensions au mm, matière, quincaillerie avec refs fournisseur
- **Plans élec/plomberie conformes NFC 15-100** : prises, disjoncteurs, évacuations, volumes de protection
- **Fiche de pose** : ordre de montage, points de vigilance, outillage recommandé
- **Liste de commande fournisseur** : exportable pour achat direct chez Blum / Hettich / Häfele

1.4 Gain business attendu

Métrique	Avant AVRA	Avec AVRA	Gain
Temps/plan technique	6 h (moy.)	1 h (30 min IA + 30 min review)	83 % de temps économisé
Plans/mois (volume type)	12-15	30+	×2 capacité commerciale
Valeur temps économisé	—	—	~3 150 €/mois/artisan
Délai client (RDV → livraison)	8-10 semaines	4-6 semaines	~40 % plus rapide
Taux d'erreurs fab.	3-5 %	<0.5 %	×10 fiabilité

NOTE : Pricing cible du module : add-on premium **79 €/mois** (20 plans inclus) + 3 €/plan supplémentaire. Coût IA réel : ~0.80 €/plan → marge brute ~80 %. Un artisan qui économise 3 000 €/mois de temps paie 79 € sans hésitation : ROI ×38.

1.5 Positionnement concurrentiel

Les concurrents directs se divisent en 3 catégories :

Catégorie	Exemples	Force	Faiblesse
Logiciels de conception historique	Polycad, WinnerFlex, KDM	Maîtrise, catalogues fournisseurs	Chers (~45k€), courbe apprentissage 6 mois
ERP/CRM métier	ProDevis, CuisiPro, Optima	Gestion commerciale OK	Pas de plan technique, UI vieillissante, pas d'IA
Acteurs IA généralistes	ChatGPT + Claude utilisés manuellement	Accessibles, innovants	Aucune spécialisation métier, pas de règles métier

La **fenêtre de tir AVRA** est claire : personne n'a encore combiné (a) une UX moderne SaaS, (b) un moteur IA spécialisé métier français, (c) une intégration complète dans un flux dossier existant (de la vente au chantier), (d) une tarification accessible aux indépendants.

1.6 Vision à 24 mois

Le Plan Technique IA est le **cheval de Troie commercial d'AVRA**. Une fois un artisan convaincu par le ROI du module (tangible dès le premier plan généré), il ne peut plus revenir en arrière — il devient dépendant d'AVRA pour sa production. Ses **corrections répétées** (fine-tuning implicite) créent un **effet réseau individuel** : le modèle apprend son style, ses conventions, ses fournisseurs préférés. Au bout de 50 plans, changer d'outil = tout re-configurer = impossible.

L'objectif à 24 mois est d'atteindre **500 cuisinistes/menusiers payants** (~40k€ MRR add-on seul) et de positionner AVRA comme **la référence française de l'ERP IA pour l'agencement sur-mesure**.

2. Glossaire métier complet

Ce glossaire est essentiel pour Claude Code et tout dev extérieur au métier. Chaque terme a une définition précise et des exemples. En cas de doute dans le code, se référer à cette section avant d'improviser.

V3 APD

Avant-Projet Définitif version 3 — le plan 3D signé par le client (particulier) avec son devis. Contient une vue 3D photoréaliste + une cotation globale + une liste des appareils électroménagers commandés. Généralement produit avec Polyboard, WinnerFlex, SketchUp ou KD Max. N'est **PAS** un document de fabrication.

DCE

Dossier de Consultation des Entreprises — le document technique interne destiné à l'atelier et aux fournisseurs. Contient vue éclatée, nomenclature, plans élec/plomb, fiche de pose. **C'est ce que nous générons.**

Relevé de mesures

Croquis (souvent manuel, parfois photo smartphone) pris sur le chantier client avant fabrication. Contient les dimensions exactes des murs (rarement aux dimensions V3), les arrivées eau (position + hauteur du sol), évacuations, prises électriques existantes, hauteur sous plafond, type de sol, nature des murs (placo, brique, béton).

Caisson

Un meuble individuel dans une cuisine. 4 familles principales : **bas** (sol jusqu'à 85-90 cm), **haut** (au-dessus du plan de travail), **colonne** (du sol au plafond, pour four/frigo encastrés), **îlot** (meuble autoportant au centre). Chaque caisson se décompose en 10-25 pièces.

Pièce

Élément unitaire composant un caisson. Exemples : fond, côté gauche, côté droit, tablette, traverse haut, traverse bas, façade, plinthe. Chaque pièce a des dimensions au mm près (longueur × largeur × épaisseur), une matière (mélaminé, massif hêtre, MDF, contreplaqué), une face décor, et éventuellement des usinages (rainure, perçage, placage de chant).

Quincaillerie

Tous les éléments non-bois d'un caisson : charnières, tiroirs, coulisses, poignées, boutons, pieds, rails LED, fonds de tiroir, paniers filaires, bouteillers, amortisseurs. Les 3 marques dominantes en France : **Blum (AT, premium)**, **Hettich (DE, milieu de gamme)**, **Häfele (DE, multi-marques)**. Chaque élément a une référence catalogue précise.

Nomenclature

Liste exhaustive de toutes les pièces + toute la quincaillerie nécessaires à la fabrication d'un projet, avec dimensions, quantités, matières, références fournisseur et prix unitaire. Document central du DCE.

Fiche de pose

Document destiné au poseur (installateur) sur chantier. Contient : ordre de pose (généralement de gauche à droite, du bas vers le haut), points de vigilance (mur non d'aplomb, arrivée d'eau à déplacer, plinthe à recouper), outillage spécifique requis, temps de pose estimé, jeux de réglage admissibles.

Plan d'implantation

Vue de dessus (plan) + vues de face (élévations, une par mur concerné) de la cuisine/agencement, avec toutes les cotes + positions appareils + positions prises/arrivées/évacuations. C'est le document de référence pour le plombier et l'électricien.

NFC 15-100

Norme française obligatoire pour les installations électriques basse tension dans les bâtiments d'habitation. Définit les volumes de protection salle de bain, le nombre minimum de prises par pièce, les sections de câbles (1.5 mm² pour 10A, 2.5 mm² pour 16A, 6 mm² pour 32A plaque induction), les dispositifs différentiels, et les types de gaines (ICTA).

Plinthe

Bandeau horizontal en bas des meubles bas, masquant les pieds réglables. Hauteur standard : 10-15 cm. S'emboîte par clips, recoupable sur chantier.

Volume de protection

Zones normatives autour d'un point d'eau (douche, baignoire, évier) où le matériel électrique doit respecter un indice de protection IP minimal. Volumes 0, 1, 2 et hors-volume avec des règles précises d'installation.

Plan de travail

Surface horizontale recouvrant les meubles bas. Matières courantes : stratifié, quartz, granit, céramique, compact HPL. Épaisseurs 38, 20 ou 12 mm. Découpes pour évier et plaque de cuisson à coter avec précision.

Crédence

Bandeau vertical entre plan de travail et meubles haut (typiquement 60-65 cm de haut). Matières : crédence verre laqué, inox, carrelage, compact HPL. Souvent oubliée des plans techniques — erreur classique.

ICTA

Isolant Cintrable Transversalement Annelé — la gaine orange/grise obligatoire pour les câbles électriques en France. Diamètres standards : 16, 20, 25, 32 mm selon section et nombre de conducteurs.

Encastrément

Surface réservée dans un meuble colonne ou bas pour y insérer un appareil électroménager (four, micro-ondes, lave-vaisselle, frigo). Chaque appareil a une fiche technique constructeur avec les dimensions exactes d'encastrément requises (souvent millimétrées).

3. User stories détaillées (Gherkin)

Les user stories ci-dessous sont rédigées au format **Gherkin** (Given / When / Then) pour être actionnables sans ambiguïté par un dev. Chaque story est numérotée (US-xx) et inclut un critère d'acceptance mesurable.

US-01 — Lancer la génération d'un plan technique

En tant que cuisiniste, je veux générer un plan technique depuis un dossier existant

Scenario :

Given un dossier AVRA en statut EN_COURS avec au moins une V3 PDF uploadée
And au moins un relevé de mesures (photo ou PDF) uploadé
When je clique sur le bouton « Générer le Plan Technique »
Then un job asynchrone est créé en base avec status PENDING
And une notification in-app m'informe que la génération a démarré
And je peux continuer à travailler sur l'app pendant la génération

Critère d'acceptance : Latence de création du job < 500 ms. Réponse HTTP 202 Accepted avec le job ID.

US-02 — Suivre l'avancement de la génération

En tant que cuisiniste, je veux voir l'avancement du traitement IA en temps réel

Scenario :

Given un job de génération en cours (status PROCESSING)
When je suis sur la page du dossier ou sur /plan-technique
Then je vois une barre de progression avec les étapes : « Extraction V3 » (0-25%), « Lecture relevé » (25-40%), « Application règles métier » (40-70%), « Génération PDF » (70-95%), « Finalisation » (95-100%)
And chaque étape a une durée estimée affichée
And si une étape dépasse son budget temps de 2x, un warning s'affiche

Critère d'acceptance : Polling SSE ou WebSocket. Pas de polling HTTP simple. Déviation < 5% vs réel.

US-03 — Recevoir une notification de fin de génération

En tant que cuisiniste, je veux être notifié dès que mon plan est prêt

Scenario :

Given un job en status PROCESSING
When le job passe à status DONE
Then je reçois une notification in-app (toast + entrée dans la cloche notifications)
And optionnellement un email avec lien vers le plan

And le PDF est téléchargeable immédiatement

Critère d'acceptance : Délai notification < 2 s après passage DONE. Email optionnel sous 30 s.

US-04 — Visualiser le plan généré

En tant que cuisiniste, je veux consulter le plan avant de l'envoyer à l'atelier

Scenario :

Given un plan généré avec status DONE

When j'ouvre la page du plan

Then je vois un viewer PDF intégré affichant les 10-15 pages

And je peux naviguer entre les pages, zoomer, télécharger

And je vois un bouton « Corriger » à côté de chaque section

Critère d'acceptance : Rendu < 2 s. PDF ≤ 10 Mo. Viewer responsive mobile + desktop.

US-05 — Corriger une dimension erronée

En tant que cuisiniste, je veux corriger manuellement une dimension mal détectée

Scenario :

Given un plan généré contenant un meuble bas détecté à 60 cm alors qu'il fait 80 cm

When je clique sur la cellule dimension du meuble dans le tableau éditable

Then je peux saisir la nouvelle valeur

And toutes les pièces dépendantes sont recalculées automatiquement (fond, tablette, traverses)

And la quincaillerie est re-matchée si nécessaire (tiroir 60 cm → tiroir 80 cm)

And je peux soit régénérer le PDF complet, soit mettre à jour uniquement les pages concernées

Critère d'acceptance : Recalcul pièces dépendantes < 500 ms. Régénération partielle < 10 s.

US-06 — Exporter la liste de commande fournisseur

En tant que cuisiniste, je veux exporter la nomenclature pour passer commande

Scenario :

Given un plan validé avec nomenclature complète

When je clique sur « Exporter commande »

Then je peux choisir le format (PDF, Excel, copie-coller)

And je peux filtrer par fournisseur (tout, Blum uniquement, Hettich uniquement)

And l'export contient : ref fournisseur, désignation, quantité, prix unitaire, total

Critère d'acceptance : Export < 3 s. Prix à jour (catalogue mis à jour < 90 jours).

US-07 — Régénérer avec un fichier corrigé

En tant que cuisiniste, si la V3 a changé, je veux régénérer sans perdre mes corrections

Scenario :

Given un plan généré v1 avec mes corrections manuelles
And une nouvelle V3 (v2) uploadée dans le dossier
When je clique sur « Régénérer avec nouvelle V3 »
Then l'IA détecte les différences V1 vs V2 (meubles ajoutés/retirés/modifiés)
And mes corrections précédentes sont appliquées automatiquement sur les éléments inchangés
And je vois un diff visuel clair des changements

Critère d'acceptance : Détection diff < 5 s. Conservation corrections pour ≥ 90% des pièces inchangées.

US-08 — Versionning des plans

En tant que cuisiniste, je veux conserver l'historique des versions du plan

Scenario :

Given plusieurs plans générés pour un même dossier
When j'ouvre l'onglet « Versions »
Then je vois la liste chronologique (v1, v2, v3...) avec date + auteur + changements
And je peux télécharger n'importe quelle version passée
And je peux comparer 2 versions côte à côte

Critère d'acceptance : Limite : 20 versions max par dossier. Au-delà, auto-archivage version la plus ancienne.

US-09 — Gérer les erreurs de génération IA

En tant que cuisiniste, je veux comprendre quand l'IA ne peut pas générer le plan

Scenario :

Given un dossier avec une V3 PDF corrompue ou illisible
When je clique sur « Générer le Plan Technique »
Then le job passe à status FAILED après 2 retries automatiques
And je vois un message clair : « La V3 n'a pas pu être lue. Vérifiez que le fichier PDF est complet et lisible »
And je peux contacter le support en 1 clic avec le contexte du job

Critère d'acceptance : Taux d'échec global < 2%. Message d'erreur compréhensible à 100%.

US-10 — Valider la facturation des plans

En tant qu'admin AVRA, je veux voir la consommation du module par workspace

Scenario :

Given un workspace avec abonnement Plan Technique IA actif

When j'ouvre le portail admin /portail-admin/plan-technique

Then je vois le nombre de plans générés ce mois-ci par utilisateur

And le coût IA associé (tokens consommés × prix unitaire)

And le montant facturable (quota inclus 20 plans, puis 3 €/plan)

Critère d'acceptance : Données rafraîchies < 30 min. Précision coût IA ± 5%.

NOTE : Stories supplémentaires US-11 à US-25 disponibles en annexe A (couvrent : détection collisions meubles, suggestion optimisation matières, mode hors-ligne, partage externe atelier via lien, support multi-langue FR/EN, etc.). Priorité v1 : US-01 à US-10.

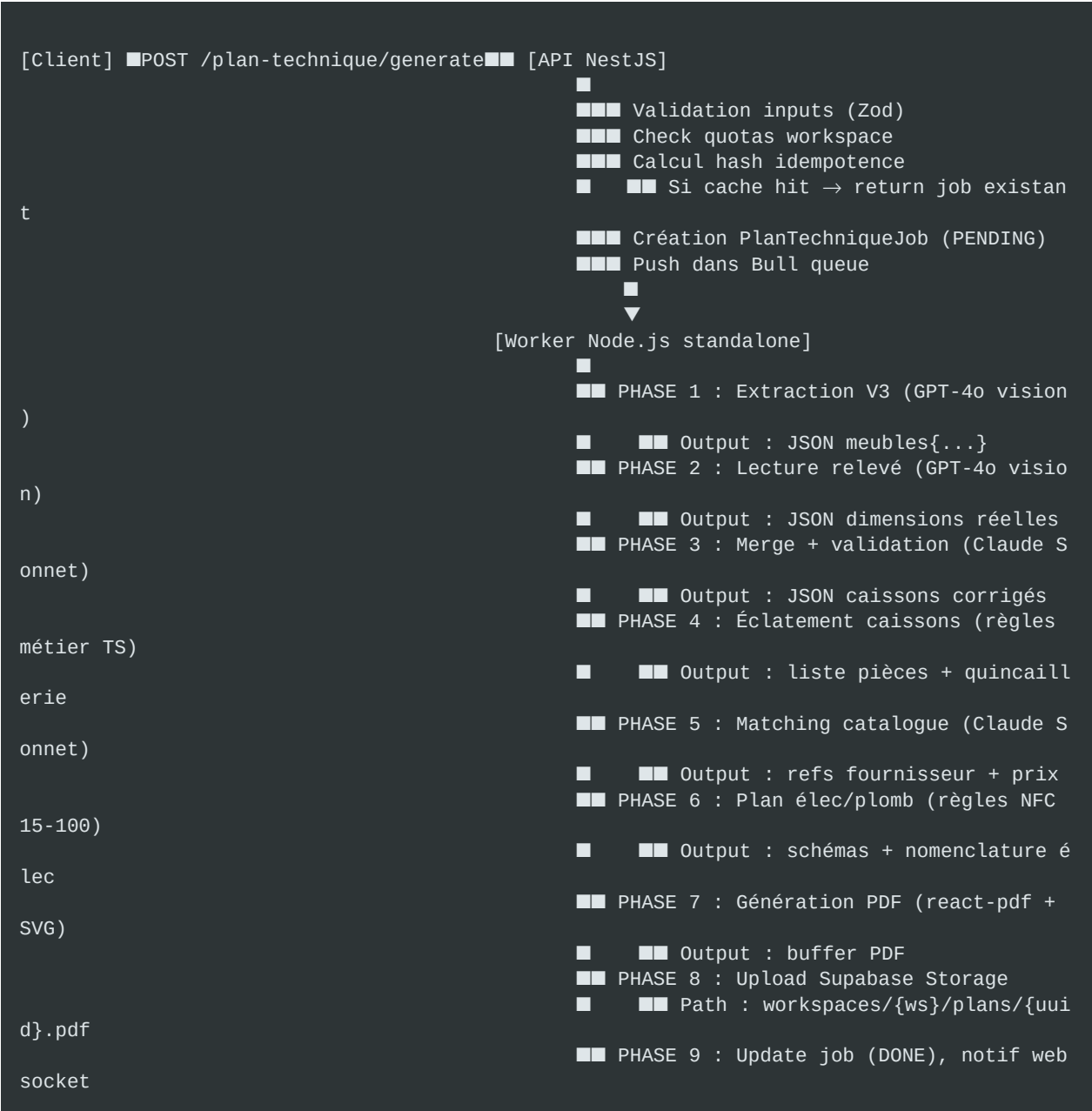
4. Architecture technique globale

4.1 Vue d'ensemble

Le module s'intègre dans la stack AVRA existante sans rupture. Les principes architecturaux directeurs sont :

- **Asynchrone par défaut** : toute génération passe par une queue Bull (Redis), jamais en sync HTTP. Permet de respecter les limites Vercel (60s max) et d'offrir une UX robuste (notif quand prêt).
- **Idempotence totale** : chaque job a un hash déterministe basé sur les inputs. Si le même user relance avec les mêmes fichiers, on retourne le résultat précédent (économie tokens IA).
- **Observabilité first-class** : chaque étape du pipeline est tracée (Sentry + métriques custom), chaque appel IA est audité (input / output / latence / coût).
- **Multi-modèle** : GPT-4o (vision), Claude Sonnet 4.6 (raisonnement), Gemini 2.0 Flash (fallback). Fallback automatique en cas d'outage provider.
- **Security by design** : même JwtAuthGuard que le reste, ownership workspace validé à chaque endpoint, service_role Supabase server-only, PDFs stockés bucket privé avec URLs signées 60 min.

4.2 Diagramme de flux (texte)



4.3 Composants à créer / modifier

Backend (NestJS)

- **apps/api/src/modules/plan-technique/** — nouveau module complet
- **plan-technique.controller.ts** — endpoints REST
- **plan-technique.service.ts** — orchestration
- **plan-technique.module.ts** — wiring NestJS
- **dto/** — DTOs Zod validés (create, update, correction)
- **guards/** — ownership + quota guards
- **apps/api/src/modules/ai-pipeline/** — nouveau, services IA partagés
- **openai.service.ts** — wrapper GPT-4o vision
- **anthropic.service.ts** — wrapper Claude Sonnet

- • gemini.service.ts — wrapper fallback
- • multi-model-orchestrator.service.ts — retry + fallback logic
- • prompts/ — prompts versionnés (V1, V2...)
- **apps/api/src/modules/caisson-engine/** — moteur de règles métier
- • caisson-types.ts — types TS (Caisson, Piece, Quincaillerie)
- • caisson-exploder.service.ts — éclatement des caissons
- • rules/ — règles par famille (bas, haut, colonne, îlot)
- • nfc-15100.service.ts — règles élec françaises
- **apps/api/src/modules/pdf-generator/** — génération PDF vectoriel
- • pdf-generator.service.ts — orchestrateur
- • templates/ — templates react-pdf par page
- • svg/ — générateurs SVG pour vue éclatée/plan impl.
- **apps/api/src/workers/plan-technique.worker.ts** — worker Bull async
- **apps/api/src/modules/catalogue/** — catalogues fournisseurs
- • blum.service.ts, hettich.service.ts, hafele.service.ts
- • catalogue.seed.ts — import initial catalogues CSV/API

Frontend (Next.js)

- **apps/web/app/(app)/plan-technique/** — nouvelle route section
- • page.tsx — dashboard plans générés
- • [dossierId]/page.tsx — vue plan + éditeur corrections
- • [dossierId]/generate/page.tsx — wizard de lancement
- **apps/web/lib/plan-technique-api.ts** — client API typé
- **apps/web/store/usePlanTechniqueStore.ts** — store Zustand
- **apps/web/components/plan-technique/** — composants dédiés
- • PdfViewer.tsx, CaissonEditor.tsx, ProgressStepper.tsx
- • QuincaillerieTable.tsx, VersionTimeline.tsx

Infrastructure

- **Queue** : Bull (déjà présent dans AVRA via le module IA photo) + Redis managé (Upstash recommandé)
- **Worker** : process Node.js séparé (Railway ou Fly.io), NON serverless. Raisons : traitement long (2-5 min), besoin de mémoire stable (~1 Go), coûts moindres qu'en Vercel
- **Storage** : bucket Supabase plans-techniques privé (comme dossier-documents)
- **Secrets** : OPENAI_API_KEY, ANTHROPIC_API_KEY, GEMINI_API_KEY, REDIS_URL — tous côté serveur uniquement

5. Schéma Prisma complet

7 nouveaux modèles à ajouter dans `prisma/schema.prisma`. Migration SQL à suivre. Relations avec les modèles existants (Workspace, Project, User) respectent les conventions AVRA (cascade delete, indexes composés).

```
model PlanTechnique {
  id          String    @id @default(cuid())
  workspaceId String
  projectId   String
  version     Int        @default(1)
  status      PlanTechniqueStatus @default(DRAFT)
  pdfStoragePath String? // path bucket Supabase
  pdfSizeBytes Int?
  generationCost Decimal? @db.Decimal(10, 4) // coût IA en EUR
  createdBy   String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  workspace      Workspace    @relation(fields: [workspaceId], references: [id], onDelete: Cascade)
  project        Project      @relation(fields: [projectId], references: [id], onDelete: Cascade)
  createdBy      User          @relation("PlanTechniqueCreatedBy", fields: [createdById], references: [id])
  jobs           PlanTechniqueJob[]
  caissons       Caisson[]
  corrections    PlanTechniqueCorrection[]
  inputSnapshot  PlanTechniqueInputSnapshot?

  @@index([workspaceId, projectId])
  @@index([projectId, version])
  @@index([createdAt])
}

enum PlanTechniqueStatus {
  DRAFT
  GENERATING
  READY
  FAILED
  ARCHIVED
}

model PlanTechniqueJob {
  id          String    @id @default(cuid())
  planId      String
  status      JobStatus @default(PENDING)
  currentPhase JobPhase? // null si PENDING/DONE/FAILED
  progressPercent Int    @default(0)
  startedAt   DateTime?
  finishedAt  DateTime?
  errorMessage String?
  errorCode   String? // IA_FAILED, V3_UNREADABLE, etc.
  retryCount  Int        @default(0)
  inputHash   String     // pour idempotence
  modelUsed   String?    // "gpt-4o" / "claude-sonnet-4-6" / "gemini-2-flash"
  totalTokens Int?
  totalCostEur Decimal? @db.Decimal(10, 6)

  plan      PlanTechnique @relation(fields: [planId], references: [id], onDelete: Cascade)
}
```

```

auditLogs      PlanTechniqueAuditLog[]

@@index([planId])
@@index([status, startedAt])
@@index([inputHash]) // lookup idempotence
}

enum JobStatus {
    PENDING
    PROCESSING
    DONE
    FAILED
    CANCELLED
}

enum JobPhase {
    EXTRACT_V3
    READ_RELEVE
    MERGE_VALIDATE
    EXPLODE_CAISSONS
    MATCH_CATALOGUE
    PLAN_ELEC_PLOMB
    GENERATE_PDF
    UPLOAD_STORAGE
    FINALIZE
}

model Caisson {
    id          String    @id @default(cuid())
    planId      String
    position    Int        // ordre d'affichage
    type        CaissonType
    name        String     // "Meuble bas 60 1 tiroir"
    widthMm     Int
    heightMm    Int
    depthMm     Int
    materialFront String   // ex: "Mélaminé blanc"
    materialCarcass String  // ex: "Mélaminé hêtre 18 mm"
    appareilId  String?    // FK vers Appareil si encastré
    positionX   Int        // position sur le plan, mm depuis origine
    positionY   Int
    wallIndex   Int        // 0=mur A, 1=mur B, etc.

    plan        PlanTechnique @relation(fields: [planId], references: [id], onDelete:
Cascade)
    pieces      CaissonPiece[]
    quincailleries CaissonQuincaillerie[]

    @@index([planId, position])
}

enum CaissonType {
    BAS_SIMPLE
    BAS_TIROIR
    BAS_ANGLE
    BAS_COLONNE_FOUR
    BAS_COLONNE_FRIGO
    HAUT_1_PORTE
    HAUT_2_PORTES
    HAUT_RELEVABLE
    ILOT
    PLACARD
    DRESSING
    BIBLIOTHEQUE

```



```

    AUTRE
}

model CaissonPiece {
    id                String    @id @default(cuid())
    caissonId         String
    pieceRole         PieceRole
    widthMm           Int
    heightMm          Int
    thicknessMm       Int
    material          String
    quantity          Int       @default(1)
    edging            String?   // placage de chant "2L" / "4L" / null
    usinage           String?   // "rainure 8mm axe 10"

    caisson           Caisson   @relation(fields: [caissonId], references: [id], onDelete: Cascade)

    @@index([caissonId])
}

enum PieceRole {
    FOND
    COTE_GAUCHE
    COTE_DROIT
    TRAVERSE_HAUT
    TRAVERSE_BAS
    TABLETTE
    FACADE
    TIROIR_FOND
    TIROIR_COTE
    TIROIR_FACE
    PLINTHE
    CREMAILLERE
    AUTRE
}

model CaissonQuincaillerie {
    id                String    @id @default(cuid())
    caissonId         String
    catalogueRef      String    // ex: "BLUM-71B3550"
    marque            String    // BLUM / HETTICH / HAFELE
    designation       String
    quantity          Int
    unitPriceEur      Decimal   @db.Decimal(8, 2)

    caisson           Caisson   @relation(fields: [caissonId], references: [id], onDelete: Cascade)

    @@index([caissonId])
    @@index([catalogueRef])
}

model PlanTechniqueCorrection {
    id                String    @id @default(cuid())
    planId            String
    userId            String
    targetType        CorrectionTarget
    targetId          String    // ID du Caisson / Piece / Quincaillerie
    fieldName         String    // "widthMm", "material", etc.
    oldValue          String
    newValue          String
    appliedAt         DateTime  @default(now())
}

```

```

    plan          PlanTechnique @relation(fields: [planId], references: [id], onDelete:
Cascade)
    user          User          @relation(fields: [userId], references: [id])

    @@index([planId, appliedAt])
}

enum CorrectionTarget {
    CAISSON
    PIECE
    QUINCAILLERIE
    PLAN_ELEC
}

model PlanTechniqueInputSnapshot {
    planId        String        @id
    v3FileUrl      String
    v3FileHash     String
    releveFileUrl  String
    releveFileHash String
    appareilsJson  Json          // snapshot des appareils commandés au moment gen

    plan          PlanTechnique @relation(fields: [planId], references: [id], onDelete
: Cascade)
}

model PlanTechniqueAuditLog {
    id            String        @id @default(cuid())
    jobId         String
    phase         JobPhase
    modelUsed      String        // "gpt-4o" etc.
    promptVersion  String        // "v1.2.3"
    tokensInput    Int
    tokensOutput   Int
    costEur        Decimal       @db.Decimal(10, 6)
    latencyMs      Int
    promptSnapshot String        @db.Text // pour rejeu
    responseRaw    Json          // sortie brute IA
    succeeded       Boolean
    errorMessage   String?
    createdAt      DateTime      @default(now())

    job           PlanTechniqueJob @relation(fields: [jobId], references: [id], onDelete
: Cascade)

    @@index([jobId, phase])
    @@index([createdAt])
    @@index([modelUsed, succeeded])
}

```

ATTENTION : Prisma shim à jour : ne pas oublier d'ajouter les 7 entrées dans `apps/api/src/types/prisma-client.d.ts` (readonly `planTechnique`, `planTechniqueJob`, `caisson`, `caissonPiec`, `caissonQuincaillerie`, `planTechniqueCorrection`, `planTechniqueInputSnapshot`, `planTechniqueAuditLog`). Sinon compile fail.

6. API Endpoints (contract)

Tous les endpoints sont montés sous `/api/v1/plan-technique`, protégés par `JwtAuthGuard`, avec `@SkipCsrf()` (cohérent avec dossier-documents). Ownership validé dans le service.

Méthode	Route	Description
POST	<code>/:projectId/generate</code>	Lancer génération
GET	<code>/:projectId/latest</code>	Plan en cours / dernier généré
GET	<code>/:projectId/versions</code>	Historique versions
GET	<code>/:planId</code>	Détail d'un plan
GET	<code>/jobs/:jobId/status</code>	Statut d'un job
GET	<code>/jobs/:jobId/stream</code>	SSE temps réel
PATCH	<code>/:planId/corrections</code>	Appliquer correction
POST	<code>/:planId/regenerate</code>	Régénérer (garder corrections)
POST	<code>/:planId/export</code>	Exporter nomenclature
DELETE	<code>/:planId</code>	Archiver (soft delete)
GET	<code>/quota</code>	Quota usage du workspace

POST `/:projectId/generate` — Lancer génération

Request : Body: { force?: boolean, strategy?: 'full' | 'incremental' }

Response : 202 Accepted, body: { jobId, planId, status: 'PENDING' }

Notes : *Quota vérifié. Idempotence via hash inputs (sauf force=true).*

GET `/:projectId/latest` — Plan en cours / dernier généré

Request : Query: ?version=latest|number

Response : 200 OK, body: { plan: PlanTechniqueDto, currentJob?: JobDto }

Notes : *Renvoie null si aucun plan pour ce projet.*

GET `/:projectId/versions` — Historique versions

Request : —

Response : 200 OK, body: { versions: PlanTechniqueSummary[] }

Notes : *Trié desc par date. Limité à 20.*

GET /:planId — Détail d'un plan

Request : —

Response : 200 OK, body: { plan, caissons, corrections, signedPdfUrl }

Notes : URL PDF signée 60 min.

GET /jobs/:jobId/status — Statut d'un job

Request : —

Response : 200 OK, body: { status, currentPhase, progressPercent, estimatedRemainingSeconds }

Notes : Polling possible, mais préférer SSE.

GET /jobs/:jobId/stream — SSE temps réel

Request : —

Response : text/event-stream

Notes : Server-Sent Events. 1 event par changement de phase ou toutes les 2 s.

PATCH /:planId/corrections — Appliquer correction

Request : Body: { targetType, targetId, fieldName, newValue, regeneratePdf: boolean }

Response : 200 OK, body: { plan updated, jobId? si regen }

Notes : Recalcul pièces dépendantes automatique.

POST /:planId/regenerate — Régénérer (garder corrections)

Request : Body: { keepCorrections: boolean }

Response : 202 Accepted, body: { jobId, newVersion }

Notes : Crée nouvelle version, n'écrase pas l'ancienne.

POST /:planId/export — Exporter nomenclature

Request : Body: { format: 'pdf' | 'excel' | 'csv', supplier?: 'BLUM' | 'HETTICH' | 'HAFELE' | 'ALL' }

Response : 200 OK avec download URL signée

Notes : Max 5 exports / min par user.

DELETE /:planId — Archiver (soft delete)

Request : —

Response : 204 No Content

Notes : *Status passe à ARCHIVED, PDF reste 90j puis purgé.*

GET /quota — Quota usage du workspace

Request : —

Response : 200 OK, body: { periodStart, periodEnd, plansUsed, plansIncluded, extraPlansPaid }

Notes : *Données calculées temps réel.*

7. Pipeline IA détaillé

7.1 Stack IA retenue

Architecture **multi-modèle avec fallback** :

Modèle	Rôle	Tokens moyens	Coût/plan
GPT-4o (OpenAI)	Vision V3 + relevé + appareils	12k in / 8k out	~0.08 €
Claude Sonnet 4.6	Raisonnement métier + matching	20k in / 15k out	~0.45 €
Gemini 2.0 Flash	Validation croisée + fallback	10k in / 5k out	~0.05 €
Total worst case	(avec retry)	—	~1.10 €
Total nominal	—	—	~0.60 €

7.2 Phase 1 — Extraction V3 (GPT-4o vision)

Input : PDF de la V3 (ou screenshot/photo). Output : JSON structuré listant chaque meuble identifié avec ses dimensions apparentes, sa position, son type.

Prompt système (extract-v3-v1.2.md)

Tu es un expert en lecture de plans de cuisine/agencement français. Tu analyses une image ou un PDF représentant une V3 APD (Avant-Projet Définitif version 3) – un plan 3D ou 2D d'une cuisine/dressing/placard avec la liste des meubles et leurs cotations.

OBJECTIF :

Extraire de manière STRICTE et DÉTERMINISTE la liste exhaustive des meubles visibles, avec leurs dimensions apparentes et leur position relative. N'invente JAMAIS une dimension : si tu n'es pas sûr, mets `"confidence": "low"` et explique dans `"notes"`.

FORMAT SORTIE (JSON STRICT, pas de markdown, pas de prose) :

```
{
  "meubles": [
    {
      "id": "M01", // ordre d'apparition gauche → droite
      "type": "BAS_SIMPLE|BAS_TIROIR|...",
      "designation": "Meuble bas 60 1 porte",
      "width_mm": 600,
      "height_mm": 850,
      "depth_mm": 560,
      "wall_index": 0, // 0 = mur principal, 1 = mur retour, etc.
      "position_x_mm": 200, // depuis angle gauche du mur
      "contient_appareil": "FOUR|FRIGO|LV|PLAQUE|HOTTE|null",
      "material_facade": "string|null",
      "confidence": "high|medium|low",
      "notes": "..."
    }
  ],
  "appareils_dectetes": ["FOUR_60", "INDUCTION_60", "HOTTE_60", ...],
  "murs": [
    { "index": 0, "longueur_mm": 3200, "hauteur_mm": 2500 }
  ],
  "warnings": ["string", ...]
}
```

RÈGLES :

1. Si tu vois un meuble d'angle, type = BAS_ANGLE (pas BAS_SIMPLE).
2. Colonne four + micro-ondes = un seul meuble type BAS_COLONNE_FOUR.
3. Ne confonds pas « meuble bas » et « meuble haut » : les meubles haut sont au-dessus du plan de travail (niveau ~90 cm), moins profonds (35 cm standard vs 56 cm pour bas).
4. Si cotation absente sur un meuble, utilise les standards français :
bas = 720/820/870 mm haut, haut = 600/720/900 mm haut, profondeur bas = 560 mm, profondeur haut = 340-370 mm.
5. Plinthe non comptée dans la hauteur meuble (15 cm sous caisson).
6. Retourne uniquement du JSON valide, AUCUN texte autour.

Exemple de sortie (cas nominal)

```
{
  "meubles": [
    {
      "id": "M01", "type": "BAS_SIMPLE", "designation": "Meuble bas 60 1 porte",
      "width_mm": 600, "height_mm": 820, "depth_mm": 560,
      "wall_index": 0, "position_x_mm": 0,
      "contient_appareil": null, "material_facade": "Chêne clair",
      "confidence": "high", "notes": ""
    },
    {
      "id": "M02", "type": "BAS_COLONNE_FOUR", "designation": "Colonne four + M0",
      "width_mm": 600, "height_mm": 2250, "depth_mm": 560,
      "wall_index": 0, "position_x_mm": 600,
      "contient_appareil": "FOUR", "material_facade": "Chêne clair",
      "confidence": "high", "notes": "Four Bosch HBG6764B1 + M0 HMG636BS1 identifiés"
    }
  ],
  "appareils_detectes": ["FOUR_60", "MICRO_ONDES_60", "INDUCTION_60", "HOTTE_60", "LV_60"],
  "murs": [
    { "index": 0, "longueur_mm": 3200, "hauteur_mm": 2500 }
  ],
  "warnings": []
}
```

Gestion des erreurs & edge cases

- **V3 illisible** (PDF scanné basse qualité) : retry avec prompt enrichi demandant OCR préalable ; si échec retry, passer en FAILED avec errorCode=V3_UNREADABLE
- **JSON malformé** : tenter de parser, si échec re-prompter avec « Ta précédente réponse n'était pas du JSON valide, voici l'erreur : {err}. Recommence »
- **Dimensions aberrantes** (meuble 5 m de large) : retry avec prompt indiquant « Les dimensions doivent être réalistes (max 1.2 m par meuble) »
- **Meubles manquants** (si visible < 80% des meubles cotés) : warning, on continue mais on flag dans le log. L'utilisateur pourra compléter manuellement en phase édition

7.3 Phase 2 — Lecture du relevé de mesures

Input : photo du croquis chantier (ou PDF). Output : JSON avec dimensions réelles des murs, positions eau/évacuation/prises existantes, nature des matériaux.

Même modèle que phase 1 (GPT-4o vision) avec un prompt dédié read-releve-v1.0.md. Voir annexe B pour le prompt complet.

7.4 Phase 3 — Fusion & validation (Claude Sonnet)

Claude Sonnet prend en entrée les outputs phase 1 + phase 2 et produit une structure corrigée : les dimensions V3 sont ajustées aux dimensions réelles du relevé, les positions sont recalculées, les collisions détectées (ex: meuble de 60 cm dans un espace de 55 cm).

Claude est meilleur que GPT-4o pour ce type de raisonnement structuré avec contraintes multiples. Prompt : merge-validate-v1.0.md (annexe B).

7.5 Phase 4-6 — Règles métier pures (TypeScript)

Phases déterministes, **pas d'IA**. Implémentation TS pure dans caisson-engine/. Voir section 8 pour le détail des règles. Budget temps : < 2 s pour les 3 phases combinées.

7.6 Phase 7 — Génération PDF

Assemblage des données (caissons + pièces + quincaillerie + plan élec) dans un PDF de 10-15 pages via react-pdf. Voir section 9 pour les templates.

7.7 Résilience & observabilité

- **Retry** : 2 tentatives automatiques par phase IA avec exponential backoff (1s, 4s). Si échec après retry, fallback sur modèle alternatif (Claude → GPT → Gemini).
- **Validation stricte** : chaque sortie IA parsée avec Zod. Si échec parse, retry avec message d'erreur en entrée pour auto-correction.
- **Audit exhaustif** : chaque appel IA enregistre dans PlanTechniqueAuditLog le prompt, la sortie brute, la latence, le coût en tokens, l'issue succès/échec.
- **Circuit breaker** : si taux d'échec > 20% sur 100 derniers appels, on désactive temporairement les nouvelles générations et on alerte Sentry + Slack admin.
- **Mode ghost** : nouvelle version d'un prompt testée en shadow sur 100 plans réels (résultats non affichés au user) avant rollout. Comparaison auto des sorties.

8. Moteur de règles métier

C'est le **cœur métier** du module. Ce qui fait qu'un plan technique est juste ou faux. Toutes les règles sont codées en TypeScript pur, testables à 100%, déterministes. Aucune IA ici — on ne peut pas se permettre d'hallucinations sur des dimensions.

8.1 Types fondamentaux

```
// caisson-engine/types.ts
export interface Dimensions {
  widthMm: number; // largeur
  heightMm: number; // hauteur
  depthMm: number; // profondeur
}

export type CaissonType =
  | 'BAS_SIMPLE' | 'BAS_TIROIR' | 'BAS_ANGLE'
  | 'BAS_COLONNE_FOUR' | 'BAS_COLONNE_FRIGO'
  | 'HAUT_1_PORTE' | 'HAUT_2_PORTES' | 'HAUT_RELEVABLE'
  | 'ILOT' | 'PLACARD' | 'DRESSING' | 'BIBLIOTHEQUE';

export type PieceRole =
  | 'FOND' | 'COTE_GAUCHE' | 'COTE_DROIT'
  | 'TRAVERSE_HAUT' | 'TRAVERSE_BAS'
  | 'TABLETTE' | 'FACADE'
  | 'TIROIR_FOND' | 'TIROIR_COTE' | 'TIROIR_FACE'
  | 'PLINTHE' | 'CREMAILLERE';

export interface Piece {
  role: PieceRole;
  dimensions: Dimensions;
  thicknessMm: number;
  material: string;
  quantity: number;
  edging?: '2L' | '4L'; // placage de chant
  usage?: string;
}

export interface Caisson {
  type: CaissonType;
  outerDimensions: Dimensions;
  materialCarcass: string;
  materialFront: string;
  pieces: Piece[];
  quincailleries: QuincaillerieItem[];
  appareil?: AppareilEncastre;
}
```

8.2 Règle : éclatement d'un meuble bas simple

Exemple complet de la logique pour le type le plus simple. Les autres types suivent le même pattern avec plus de pièces/quincaillerie.

```
// caisson-engine/rules/bas-simple.ts
const THICKNESS_MM = 18; // épaisseur standard panneau
const BACK_THICKNESS_MM = 8;
```

```

const DEPTH_STANDARD_BAS = 560; // profondeur Blum
const PLINTH_HEIGHT_MM = 150;

export function explodeBasSimple(caisson: CaissonInput): Caisson {
  const { widthMm, heightMm, depthMm } = caisson.outerDimensions;
  const innerWidth = widthMm - 2 * THICKNESS_MM;
  const innerHeight = heightMm - 2 * THICKNESS_MM - PLINTH_HEIGHT_MM;
  const innerDepth = depthMm - BACK_THICKNESS_MM;

  const pieces: Piece[] = [
    // 2 côtés
    { role: 'COTE_GAUCHE', thicknessMm: THICKNESS_MM,
      dimensions: { widthMm: THICKNESS_MM, heightMm: heightMm - PLINTH_HEIGHT_MM,
        depthMm: depthMm },
      material: caisson.materialCarcass, quantity: 1, edging: '2L' },
    { role: 'COTE_DROIT', thicknessMm: THICKNESS_MM,
      dimensions: { widthMm: THICKNESS_MM, heightMm: heightMm - PLINTH_HEIGHT_MM,
        depthMm: depthMm },
      material: caisson.materialCarcass, quantity: 1, edging: '2L' },
    // Fond (arrière)
    { role: 'FOND', thicknessMm: BACK_THICKNESS_MM,
      dimensions: { widthMm: innerWidth, heightMm: innerHeight + 2 * THICKNESS_MM,
        depthMm: BACK_THICKNESS_MM },
      material: caisson.materialCarcass, quantity: 1 },
    // Traverses
    { role: 'TRAVERSE_HAUT', thicknessMm: THICKNESS_MM,
      dimensions: { widthMm: innerWidth, heightMm: THICKNESS_MM, depthMm: 100 },
      material: caisson.materialCarcass, quantity: 1 },
    { role: 'TRAVERSE_BAS', thicknessMm: THICKNESS_MM,
      dimensions: { widthMm: innerWidth, heightMm: THICKNESS_MM, depthMm: innerDepth },
      material: caisson.materialCarcass, quantity: 1 },
    // Tablette intermédiaire
    { role: 'TABLETTE', thicknessMm: THICKNESS_MM,
      dimensions: { widthMm: innerWidth - 4, heightMm: THICKNESS_MM,
        depthMm: innerDepth - 10 },
      material: caisson.materialCarcass, quantity: 1, edging: '2L' },
    // Façade
    { role: 'FACADE', thicknessMm: 19,
      dimensions: { widthMm: widthMm - 4, heightMm: heightMm - PLINTH_HEIGHT_MM - 4,
        depthMm: 19 },
      material: caisson.materialFront, quantity: 1, edging: '4L' },
    // Plinthe (recoupable)
    { role: 'PLINTHE', thicknessMm: THICKNESS_MM,
      dimensions: { widthMm: widthMm, heightMm: PLINTH_HEIGHT_MM, depthMm: THICKNESS_MM
    },
      material: caisson.materialFront, quantity: 1, edging: '2L' },
  ];

  const quincailleries = [
    // 2 charnières Blum clip-top 110°
    { catalogueRef: 'BLUM-71B3550', marque: 'BLUM',
      designation: 'Charnière clip-top 110° soft-close',
      quantity: 2, unitPriceEur: 4.20 },
    // 1 poignée (défaut - user peut override)
    { catalogueRef: 'GENERIC-HZ247',
      designation: 'Poignée barre 128mm alu brossé',
      quantity: 1, unitPriceEur: 3.50 },
    // 4 pieds réglables
    { catalogueRef: 'BLUM-T51.1901',
      designation: 'Pied réglable 100-120mm',
      quantity: 4, unitPriceEur: 2.80 },
    // 2 cremailières pour tablette
    { catalogueRef: 'HAFELE-282.04.721',
      designation: 'Taquet tablette D5mm nickelé',

```

```
    quantity: 4, unitPriceEur: 0.15 },
  ];

  return { ...caisson, pieces, quincailleries };
}
```

8.3 Règles NFC 15-100 (extrait)

Règles électriques françaises obligatoires. Base de règles dans nfc-15100.service.ts. Extrait des règles les plus courantes :

Appareil	Section câble	Protection	Gaine
Plaque induction	6 mm² cuivre	Disjoncteur 32A + différentiel 30mA	AICTA 25
Four encastré	2.5 mm²	Disjoncteur 20A	ICTA 20
Hotte	1.5 mm²	Disjoncteur 16A	ICTA 16
Lave-vaisselle	2.5 mm²	Disjoncteur 20A + 30mA	ICTA 20
Frigo encastré	2.5 mm²	Disjoncteur 16A	ICTA 20
Éclairage LED	1.5 mm²	Disjoncteur 10A + driver 24V	ICTA 16
Prises plan de travail	2.5 mm²	Disjoncteur 20A (6 prises max)	ICTA 20

Règles de placement des prises :

- Minimum 3 prises 16A sur le plan de travail, espacées de 40 cm min
- Hauteur standard 1100 mm depuis sol fini (entre plan de travail et meubles haut)
- Interdit : prise dans volume 0/1 de salle de bain (volume autour douche/baignoire)
- Prise lave-vaisselle : derrière le meuble, hauteur 200 mm du sol fini, accessible
- Prise plaque : derrière colonne four OU dans boîte de connexion murale (jamais directement dans meuble)
- Prise hotte : au plafond au-dessus de la hotte, avec boîte de dérivation DCL

8.4 Validation croisée

Après éclatement, valider la cohérence globale avant génération PDF :

- **Somme largeurs caissons ≤ longueur mur** (avec tolérance 10 mm pour jeux de pose)
- **Aucune collision avec portes/fenêtres** du relevé
- **Appareils encastrés respectent les dimensions constructeur** (base de données interne avec 200+ appareils courants Bosch, Neff, AEG, Siemens...)
- **Arrivée eau accessible** depuis le meuble évier (tolérance 400 mm)
- **Nombre minimum de prises** conforme NFC 15-100 (3 minimum en cuisine > 4 m²)

ATTENTION : En cas d'échec de validation croisée, le plan est généré quand même (status=READY), mais les warnings sont affichés en rouge dans le PDF + dans l'UI. Le user peut corriger manuellement ou régénérer avec données rectifiées.

9. Génération PDF (react-pdf + SVG)

9.1 Stack retenue

@react-pdf/renderer pour la structure + typographie, @svgrdotjs/svg.js (serveur) pour les schémas 2D complexes (vue éclatée, plan d'implantation).

Justifications :

- react-pdf : syntaxe JSX, polices custom embarquées (Inter + JetBrains Mono pour chiffres techniques), compatible Node serverless (pas de headless Chrome)
- SVG côté serveur : génération programmatique des schémas, embeddable dans PDF, précision vectorielle millimétrique
- Performance : un PDF de 15 pages généré en ~3-5 s (vs 15-20 s avec Puppeteer)
- Utilisé en prod par : Notion (exports), Linear (rapports), Stripe (factures)

9.2 Structure du PDF (15 pages type)

Page	Contenu
1	Couverture (logo AVRA, nom dossier, client, date, version plan)
2	Sommaire + note de lecture
3-4	Plan d'implantation (vue dessus + élévations par mur, cotées)
5	Synthèse meubles (tableau récap : nom, dim, matière, appareil encastré)
6-10	Vue éclatée par caisson (1 page = 1 caisson avec SVG 3D + nomenclature pièces)
11-12	Plan électrique conforme NFC 15-100 (schéma + tableau disjoncteurs)
13	Plan plomberie (arrivées/évacuations, section tuyauterie)
14	Récapitulatif quincaillerie (tableau Blum/Hettich/Hafele avec refs + prix)
15	Fiche de pose (ordre de montage, points de vigilance, outillage)

9.3 Template de page : vue éclatée caisson

```
// pdf-generator/templates/CaissonDetailPage.tsx
import { Page, Text, View, StyleSheet, Svg, G, Path, Rect } from '@react-pdf/renderer';
import { generateEclateSvg } from '../svg/eclate-generator';

const styles = StyleSheet.create({
  page: { paddingTop: 40, paddingBottom: 40, paddingHorizontal: 30,
    fontFamily: 'Inter', fontSize: 10 },
  header: { borderBottomWidth: 2, borderBottomColor: '#304035',
    paddingBottom: 8, marginBottom: 16 },
  title: { fontSize: 14, fontWeight: 'bold', color: '#304035' },
  subtitle: { fontSize: 10, color: '#a67749', marginTop: 2 },
  eclateContainer: { width: '100%', height: 280, marginBottom: 14 },
  nomenclatureTitle: { fontSize: 11, fontWeight: 'bold', marginBottom: 6,
    color: '#304035' },
  table: { display: 'table', width: 'auto', borderStyle: 'solid',
    borderWidth: 0.5, borderColor: '#304035' },
  tableRow: { flexDirection: 'row' },
  tableCellHeader: { backgroundColor: '#304035', color: '#fff',
    padding: 4, fontSize: 9, fontWeight: 'bold' },
  tableCell: { padding: 4, fontSize: 8.5, borderBottomWidth: 0.5,
    borderBottomColor: '#dfe6e9' },
});

export const CaissonDetailPage: React.FC<{ caisson: Caisson }> = ({ caisson }) => {
  const eclateSvg = generateEclateSvg(caisson); // retourne un SVG string

  return (
    <Page size="A4" style={styles.page}>
      <View style={styles.header}>
        <Text style={styles.title}>
          Caisson {caisson.position.toString().padStart(2, '0')} –
          {caisson.designation}
        </Text>
        <Text style={styles.subtitle}>
          {caisson.widthMm} × {caisson.heightMm} × {caisson.depthMm} mm
          · {caisson.materialCarcass}
        </Text>
      </View>

      <View style={styles.eclateContainer}>
        {/* SVG de la vue éclatée */}
        <Svg viewBox="0 0 800 400">
          {/* Inject le SVG généré – react-pdf supporte <G>, <Path>, <Rect>, <Line>, <Text> */}
          {renderSvgElements(eclateSvg)}
        </Svg>
      </View>

      <Text style={styles.nomenclatureTitle}>Nomenclature pièces</Text>
      <View style={styles.table}>
        <View style={styles.tableRow}>
          <Text style={styles.tableCellHeader, { width: '25%' }}>Pièce</Text>
          <Text style={styles.tableCellHeader, { width: '20%' }}>L × H × P (mm)</Text>
          <Text style={styles.tableCellHeader, { width: '25%' }}>Matière</Text>
          <Text style={styles.tableCellHeader, { width: '10%' }}>Qté</Text>
          <Text style={styles.tableCellHeader, { width: '20%' }}>Chants / Usinage</Text>
        </View>
        {caisson.pieces.map((p, i) => (
          <View key={i} style={styles.tableRow}>
            <Text style={styles.tableCell, { width: '25%' }}>

```

```

        {formatPieceRole(p.role)}
      </Text>
      <Text style=[[styles.tableCell, { width: '20%' }]]>
        {p.dimensions.widthMm} × {p.dimensions.heightMm} × {p.thicknessMm}
      </Text>
      <Text style=[[styles.tableCell, { width: '25%' }]]>{p.material}</Text>
      <Text style=[[styles.tableCell, { width: '10%' }]]>{p.quantity}</Text>
      <Text style=[[styles.tableCell, { width: '20%' }]]>
        {p.edging ?? ''}{p.usinage ? ' ' + p.usinage : ''}
      </Text>
    </View>
  )}
</View>
</Page>
);
};

```

9.4 Qualité impression

- **Polices embarquées** : Inter (texte), JetBrains Mono (cotes techniques), FontAwesome (icônes)
- **Résolution SVG** : vectoriel pur, zero perte à l'impression A3
- **Couleurs** : palette AVRA (vert #304035, or #a67749) + gris techniques pour les schémas
- **Marges normées** : 2 cm partout (imprimable sur toutes imprimantes)
- **Compatibilité** : PDF/A-1b pour archivage long terme (optionnel, activable par user)

10. UI / UX frontend

10.1 Entrée dans la sidebar

Nouvel item dans la sidebar principale, sous « Intervenants », avec l'icône Sparkles de lucide-react + badge « Premium ».

```
// apps/web/components/layout/Sidebar.tsx (ajout)
{
  href: '/plan-technique',
  label: 'Plan Technique IA',
  icon: Sparkles,
  badge: 'Premium',
  badgeColor: '#a67749',
  requiresFeatureFlag: 'plan_technique_ia',
}
```

10.2 Écran principal /plan-technique

Vue liste des derniers plans générés, tous dossiers confondus. Similaire au dashboard dossiers mais filtré sur les projets ayant au moins un plan.

- Carte stats haut : plans générés ce mois, quota restant, coût IA consommé
- Liste des dossiers ayant un plan (tri par dernière génération desc)
- Chaque card : nom client, version courante, status (DRAFT/READY/GENERATING), preview miniature PDF
- Bouton « Nouveau plan » → ouvre le wizard depuis un dossier existant

10.3 Wizard de génération (4 étapes)

Étape	Contenu UI	Validation
1. Sélection dossier	Dropdown des dossiers du workspace avec statut EN_COURS.	Dossier doit exister + client ownership
2. Upload V3	Zone drop + bouton browse. Preview PDF / image. Check taille < 25 Mo. Support obligatoire MIME whitelisted.	25 Mo. Support obligatoire MIME whitelisted
3. Upload relevé	Même UX que V3. Optionnel mais fortement recommandé (warning absent si génération possible mais diminue la probabilité de succès)	Signatures générées
4. Confirmation	Récap inputs + estimation temps (2-5 min) + coût (1 plan décompté du quota).	CTA « Générer maintenant »

10.4 Écran suivi job (temps réel)

Après clic « Générer », redirection vers /plan-technique/:planId avec vue stepper animée. Connexion SSE au serveur pour updates temps réel.

- Stepper vertical avec 9 phases (voir section 7.2). Phase courante animée (pulse).
- Phases passées : check vert + durée réelle. Phases futures : grisées + durée estimée.
- Si FAILED : encart rouge avec message d'erreur + bouton retry + bouton contacter support (pré-rempli avec ID job).
- Si DONE : toast success + redirection auto vers le viewer PDF.

10.5 Viewer + éditeur

Écran principal une fois le plan généré. Layout 2 colonnes :

- **Colonne gauche (60%)** : viewer PDF intégré (react-pdf-viewer). Navigation pages, zoom, download, fullscreen.
- **Colonne droite (40%)** : onglets « Caissons », « Nomenclature », « Électrique », « Versions ». Chaque onglet = tableau éditables correspondant au contenu du PDF.
- **Drag pour resize** entre colonnes. Persistance largeur en localStorage.
- **Mode split** : sur écran large, le viewer peut être sur une autre fenêtre pour suivre les changements en direct.

10.6 Éditeur de corrections

Chaque cellule des tableaux est éditables inline. Feedback visuel immédiat :

- Cellule survolée : highlight subtil
- Cellule éditée : bord or, indicateur « modifié » en haut de ligne
- Recalcul automatique pièces dépendantes en ~500 ms avec feedback toast
- Barre d'action en bas : « Régénérer PDF » (actif si modifs non-sauvées), « Annuler toutes les modifs » (revert)
- Historique des corrections dans l'onglet Versions (qui a changé quoi, quand)

11. Stratégie de tests (Gold Standard)

11.1 Pyramide

Inspirée des best practices Google / Stripe. Inversée intentionnellement pour ce module critique : plus d'unit que d'E2E.

Niveau	Framework	Couverture cible	Nombre cible
Unit	Vitest	100 % règles métier, 90 % services	~800 tests
Integration	Vitest + supertest	80 % endpoints API	~150 tests
Contract	Vitest + Zod	100 % prompts IA (snapshot)	~30 tests
E2E Frontend	Playwright	Parcours critiques user	~20 tests
Non-régression IA	Custom runner	50 V3 golden dataset	~50 tests

11.2 Unit tests : exemple règles métier

```
// caisson-engine/rules/__tests__/bas-simple.spec.ts
import { describe, it, expect } from 'vitest';
import { explodeBasSimple } from '../bas-simple';

describe('explodeBasSimple', () => {
  it('génère 8 pièces standard pour un meuble 60x82x56', () => {
    const result = explodeBasSimple({
      outerDimensions: { widthMm: 600, heightMm: 820, depthMm: 560 },
      materialCarcass: 'Mélaminé hêtre 18mm',
      materialFront: 'Chêne clair',
    });
    expect(result.pieces).toHaveLength(8);
    expect(result.pieces.filter(p => p.role === 'COTE_GAUCHE')).toHaveLength(1);
    expect(result.pieces.filter(p => p.role === 'FACADE')).toHaveLength(1);
  });

  it('calcule correctement les dimensions côté gauche (hauteur - plinthe)', () => {
    const result = explodeBasSimple({
      outerDimensions: { widthMm: 600, heightMm: 820, depthMm: 560 },
      materialCarcass: 'Mélaminé',
      materialFront: 'Chêne',
    });
    const coteGauche = result.pieces.find(p => p.role === 'COTE_GAUCHE')!;
    expect(coteGauche.dimensions.heightMm).toBe(820 - 150); // 670 mm
    expect(coteGauche.dimensions.depthMm).toBe(560);
    expect(coteGauche.thicknessMm).toBe(18);
  });

  it('inclut les 2 charnières Blum 71B3550', () => {
    const result = explodeBasSimple({
      outerDimensions: { widthMm: 600, heightMm: 820, depthMm: 560 },
      materialCarcass: 'Mélaminé', materialFront: 'Chêne',
    });
    const charniere = result.quincailleries.find(
      q => q.catalogueRef === 'BLUM-71B3550'
    );
    expect(charniere).toBeDefined();
    expect(charniere!.quantity).toBe(2);
  });

  // Edge cases
  it('refuse des dimensions aberrantes (width > 1200mm)', () => {
    expect(() => explodeBasSimple({
      outerDimensions: { widthMm: 1500, heightMm: 820, depthMm: 560 },
      materialCarcass: 'Mélaminé', materialFront: 'Chêne',
    })).toThrow('Width must be between 200 and 1200 mm');
  });
});
```

11.3 Non-régression IA : golden dataset

50 V3 réelles (anonymisées) fournies par les bêta-testeurs + 50 attendus JSON validés manuellement. À chaque déploiement, on re-exécute le pipeline sur les 50 et on compare avec les outputs attendus. Tolérance : max 5% de divergence globale.

```
// tests/ia-regression/runner.ts
for (const fixture of goldenDataset) {
```

```
const actual = await runExtractV3Pipeline(fixture.input);
const similarity = computeSimilarity(actual, fixture.expectedOutput);
if (similarity < 0.95) {
  console.error(`REGRESSION on fixture ${fixture.id}: ${similarity}`);
  failed++;
}
}

if (failed / goldenDataset.length > 0.05) {
  throw new Error('IA regression threshold exceeded: >5% divergence');
}
```

11.4 Mode ghost (A/B automatique)

Avant chaque déploiement d'une nouvelle version de prompt, on la fait tourner en shadow sur 100 plans réels générés récemment par les users. On compare avec l'ancienne version.

- Si similarité > 95 % → déploiement auto approuvé
- Si 90-95 % → alerte Slack, review humaine requise
- Si < 90 % → blocage, investigation obligatoire
- Tous les résultats stockés dans PlanTechniqueAuditLog avec flag isGhost=true

12. Observabilité & Ops production

12.1 Stack observabilité

- **Sentry** (déjà en place AVRA) : erreurs, stack traces, breadcrumbs
- **Logs structurés** : Pino (Node.js) avec output JSON, agrégés sur Vercel
- **Métriques** : Prometheus-compatible endpoint `/api/v1/metrics` scrapé par Grafana Cloud
- **Alerting** : Grafana + Slack (channel #avra-alerts)
- **Dashboard métier** : Supabase (SQL direct) + Metabase (dashboards users)

12.2 Métriques clés à tracker

Métrique	Seuil warning	Seuil critical
Taux de succès génération	< 90 %	< 80 %
Durée moyenne pipeline	> 3 min	> 6 min
Coût IA moyen / plan	> 1.2 €	> 2 €
Queue depth (jobs PENDING)	> 20	> 50
Taux retry IA	> 15 %	> 30 %
Erreurs 5xx endpoints	> 2 %	> 5 %
Latence p95 API	> 1 s	> 3 s
Ghost vs prod similarity	< 95 %	< 90 %

12.3 Dashboard admin (interne AVRA)

Nouvelle section `/portail-admin/plan-technique` avec :

- Liste des 100 derniers jobs, filtrable par status / workspace / model
- Vue détail d'un job : prompts exacts utilisés, outputs IA bruts, corrections user, PDF final
- Rejeu d'un job : bouton « Re-run » qui recrée un job avec les mêmes inputs (utile pour debug)
- Stats consommation par workspace : plans / mois, coût IA, marge brute
- Feature flag manuel par user : on peut activer/désactiver le module pour un workspace spécifique depuis le dashboard (utile pour bêta-testeurs)

12.4 Runbooks

3 runbooks à documenter dans le repo (`docs/runbooks/plan-technique/`) :

- **runbook-01-ia-provider-down.md** — Que faire si OpenAI/Anthropic est down
- **runbook-02-queue-saturated.md** — Que faire si Bull queue > 100 jobs
- **runbook-03-regression-detected.md** — Procédure rollback si qualité dégradée

13. Sécurité, audit trail & conformité

13.1 Security checklist

- **Auth** : JwtAuthGuard partout (cohérent avec dossier-documents)
- **Ownership** : chaque endpoint vérifie que le projet appartient au workspace du user
- **Rate limiting** : 10 générations / heure / user max (Throttler NestJS)
- **Quotas** : quota mensuel par workspace selon abonnement
- **Isolation fichiers** : path storage = workspaces/{ws}/plans/{uuid}.pdf (non-devinable)
- **URLs signées 60 min** : impossible de leak un lien PDF permanent
- **PII** : les données client (nom, adresse) ne sont JAMAIS envoyées aux providers IA. Anonymisation au niveau du prompt (remplacement des données personnelles par des tokens)
- **Secrets server-only** : OPENAI_API_KEY, ANTHROPIC_API_KEY jamais exposés client
- **CSP** : ajout de api.openai.com et api.anthropic.com dans connect-src si appels client (non recommandé pour ce module, tout server-side)

13.2 Audit trail full

Chaque plan généré stocke de manière immuable (pas de UPDATE autorisé) :

- Hash SHA-256 des inputs (V3, relevé, appareils au moment de la gen)
- Version exacte de chaque prompt utilisé (ex: extract-v3-v1.2.3)
- Outputs bruts de chaque appel IA (avant parsing / validation)
- Toutes les corrections utilisateur avec horodatage + identité
- Version du moteur de règles métier utilisée

NOTE : Rétention 5 ans minimum. Permet de rejouer un plan 3 ans après pour comprendre pourquoi telle décision a été prise, en cas de litige client. Indispensable pour défendre AVRA face à un cuisiniste qui accuse le module d'une erreur de production.

13.3 Conformité RGPD

- Les V3 / relevés uploadés peuvent contenir des données personnelles (nom client sur couverture). Traitement nécessaire → base légale : exécution du contrat (art. 6.1.b)
- Anonymisation avant envoi IA : nom client remplacé par « CLIENT_XXX »
- Droit à l'effacement : endpoint DELETE /:planId supprime PDF + audit logs associés (sauf si obligation légale de conservation)
- DPA Anthropic : signé (Anthropic utilise Zero Data Retention — les prompts ne servent pas à l'entraînement)
- DPA OpenAI : signé (mode Enterprise avec opt-out training)

14. Plan de livraison 4-6 mois

14.1 Vue d'ensemble

Livraison **Big Bang** à ~20 semaines avec milestones internes aux semaines 4, 8, 12, 16. Pas de release partielle externe : le module n'est activé pour les users qu'à la fin (flag `FEATURE_PLAN_TECHNIQUE`).

Semaine 1-4 — Foundations

- Schéma Prisma complet + migrations
- Module NestJS plan-technique avec endpoints stubs
- Module ai-pipeline avec wrappers OpenAI/Anthropic/Gemini + tests unitaires
- Worker Bull + Redis setup (sandbox dev)
- Shell UI /plan-technique placeholder
- Feature flag `plan_technique_ia` en place (OFF par défaut)

→ **Milestone M1 : endpoint `POST /generate` répond 202, mais le worker logue seulement**

Semaine 5-8 — Pipeline IA

- Phase 1 (Extract V3) fonctionnelle + 10 tests fixtures
- Phase 2 (Read Relevé) fonctionnelle + 10 tests
- Phase 3 (Merge validate) fonctionnelle + 10 tests
- Intégration Anthropic + OpenAI + fallback Gemini
- Prompts versionnés dans Git (`prompts/*.md`)
- Audit logging complet

→ **M2 : le worker produit un JSON structuré cohérent pour 8/10 cuisines test**

Semaine 9-12 — Moteur métier + PDF

- Règles éclatement caissons (8 types min) + 80 tests unitaires
- Règles NFC 15-100 élec + 30 tests
- Règles plomberie basiques
- Catalogues Blum/Hettich/Hafele importés (CSV) + matching
- Templates react-pdf pour 15 pages type
- SVG generator pour vue éclatée + plan d'implantation

→ **M3 : un PDF complet est généré bout en bout pour 1 cuisine test simple**

Semaine 13-16 — Frontend + UX

- Wizard génération 4 étapes
- Écran suivi job SSE temps réel
- Viewer PDF + éditeur tableaux (caissons/nomenclature)
- Gestion corrections + recalcul pièces dépendantes
- Écran versions + diff entre versions
- Dashboard admin /portail-admin/plan-technique

→ **M4 : un user peut générer, consulter, corriger, re-générer un plan de bout en bout**

Semaine 17-20 — Hardening + Beta closed

- Tests E2E Playwright (20 scénarios)
- Golden dataset 50 V3 réelles + runner non-régression
- Mode ghost + A/B pour déploiement prompts
- Observabilité complète (Sentry + Grafana + alertes Slack)
- Runbooks documentés
- Beta fermée avec Cassandra + 5 cuisinistes sélectionnés
- Itération sur feedback (2-3 cycles)

→ **M5 (Go/No-Go) : si 8/10 cuisinistes bêta disent « je paierais pour ça », GO release. Sinon, pivot ou extension.**

14.2 Équipe recommandée

Rôle	Temps	Responsabilité
Dev full-stack senior	Full-time 20 sem	Backend + Frontend + IA pipeline
Ingénieur ML / prompt engineer	Mi-temps 10 sem (sem 5-14)	Prompts, fine-tuning, évaluation qualité
Consultant métier (cuisiniste)	Mi-temps 6 sem (sem 9-14)	Valide les règles métier + golden dataset
Designer produit	~2 sem cumulées	Wireframes wizard + éditeur

15. Pricing & business model

15.1 Plans tarifaires

Plan	Prix/mois	Plans inclus	Plan supplémentaire	Cible
Découverte	Gratuit	2 plans/mois	—	Trial 30 jours
Starter	49 €	8 plans/mois	5 €/plan	Artisan <10 dossiers/mois
Pro	79 €	20 plans/mois	3 €/plan	Artisan actif (cible principale)
Scale	199 €	60 plans/mois	2 €/plan	Grosse PME / multi-show

15.2 Unit economics (plan Pro)

Poste	Montant/mois (20 plans)
Revenu brut abonnement Pro	79 €
Coût IA (20 × 0.80 €)	-16 €
Coût infra (prorata Vercel + Redis + Supabase)	-3 €
Coût Stripe (1.4 % + 0.25 €)	-1.35 €
Marge brute	~58.65 € (74 %)

15.3 Stratégie de pricing

- **Anchoring** : Pro à 79 € est l'offre mise en avant (best value). Découverte (gratuit) pour essayer, Scale (199 €) pour ancrer le haut.
- **Billing mensuel** : pas d'engagement, cassable en 1 clic. Critique pour convertir des artisans méfiants des SaaS.
- **Facturation à l'usage au-delà du quota** : facture Stripe séparée à J+30 pour les plans supplémentaires consommés. Évite le blocage UX.
- **Pricing display** : afficher « Économisez 3 000 €/mois de temps bureau d'études » à côté du prix, ancrer le ROI.

15.4 Projection revenue (conservateur)

Horizon	Users payants	Mix plans (%)	MRR total
M6 (post-launch)	30	50 Pro / 40 Starter / 10 Scale	~2 500 €
M12	120	60 / 30 / 10	~11 000 €
M18	280	65 / 25 / 10	~26 000 €
M24	500	65 / 25 / 10	~47 000 €

16. Risques & mitigation

ID	Risque	Impact	Probabilité
R-01	Qualité IA insuffisante (hallucinations sur dimensions)	Critique	Moyen
R-02	Coût IA dépasse la marge	Élevé	Faible
R-03	Catalogue fournisseur obsolète	Moyen	Élevé
R-04	Refus adoption par les bêta-testeurs	Critique	Moyen
R-05	Cuisiniste utilise un plan erroné en prod → litige client	Critique	Faible
R-06	Provider IA (OpenAI/Anthropic) down	Élevé	Faible
R-07	Dépassement délais (>20 semaines)	Moyen	Élevé
R-08	Complexité règles métier sous-estimée	Élevé	Moyen
R-09	Performance génération > 5 min	Moyen	Faible
R-10	Adoption concurrente (un autre SaaS shippe avant AVRA)	Moyen	Faible

R-01 — Qualité IA insuffisante (hallucinations sur dimensions) (impact critique, probabilité moyen)

Mitigation : Multi-modèle avec validation croisée. Golden dataset 50 V3 en non-régression. Mode ghost avant déploiement. Seuil min 95 % similarité.

R-02 — Coût IA dépasse la marge (impact élevé, probabilité faible)

Mitigation : Cache idempotence. Limite tokens stricte par phase. Alerting sur coût moyen > seuil. Fallback Gemini Flash (5× moins cher) si volume.

R-03 — Catalogue fournisseur obsolète (impact moyen, probabilité élevé)

Mitigation : Script sync hebdomadaire depuis sources Blum/Hettich. Warning dans PDF si ref > 90j. Permettre override manuel par user.

R-04 — Refus adoption par les bêta-testeurs (impact critique, probabilité moyen)

Mitigation : Shadowing terrain 2 semaines avant code. Bêta fermée 5-10 users pour itération. Go/No-Go M5 sur base retours réels.

R-05 — Cuisiniste utilise un plan erroné en prod → litige client (impact critique, probabilité faible)

Mitigation : Disclaimer dans PDF + UI « Plan assisté par IA, validation humaine obligatoire avant fabrication ». Audit trail full pour défense juridique. Assurance RC Pro à souscrire.

R-06 — Provider IA (OpenAI/Anthropic) down (impact élevé, probabilité faible)

Mitigation : Multi-provider avec fallback auto. Circuit breaker. Queue persistante → jobs repris quand provider revient.

R-07 — Dépassement délais (>20 semaines) (impact moyen, probabilité élevé)

Mitigation : Milestones fermes. Fonctionnalités non-critiques (SVG 3D, plans plomb avancés) retirables si retard. Prio MoSCoW sur chaque sprint.

R-08 — Complexité règles métier sous-estimée (impact élevé, probabilité moyen)

Mitigation : Consultant cuisiniste mi-temps dès S9. Commencer par 3-4 types de caisson les plus courants, étendre après. Couverture à 80% du besoin = acceptable v1.

R-09 — Performance génération > 5 min (impact moyen, probabilité faible)

Mitigation : Parallélisation phases 1-2 (possibles en parallèle). Cache agressif matching catalogue. Fallback Gemini Flash pour phase validation rapide.

R-10 — Adoption concurrente (un autre SaaS shippe avant AVRA) (impact moyen, probabilité faible)

Mitigation : Peu probable à court terme (tech stack complexe à assembler). Moat AVRA : données internes dossiers + intégration UX + corrections user apprises.

17. Annexes

A. User stories complémentaires (US-11 à US-25)

- US-11 : Détection automatique de collisions meubles avec murs non d'aplomb
- US-12 : Suggestion optimisation chutes matière (réduire déchets mélaminé)
- US-13 : Mode hors-ligne (consultation plans déjà générés sans connexion)
- US-14 : Partage externe via lien sécurisé vers l'atelier partenaire
- US-15 : Support multi-langue FR / EN pour users suisses/belges
- US-16 : Import V3 depuis Polyboard/WinnerFlex natif (parser dédié)
- US-17 : Export DXF pour CNC atelier (phase 2, hors v1)
- US-18 : Signatures multiples sur plan technique (artisan + atelier valide)
- US-19 : Clonage plan depuis un autre projet similaire
- US-20 : Calcul empreinte carbone du projet (bonus vert)
- US-21 : Suggestion alternatives moins chères par l'IA
- US-22 : Détection conformité PMR (accessibilité handicapés)
- US-23 : Génération bon de commande fournisseur direct (EDI)
- US-24 : Intégration comptabilité (export EBP/Sage)
- US-25 : Notifications push mobile quand plan prêt

B. Prompts IA complets (versionning)

Les 7 prompts détaillés disponibles dans `apps/api/src/modules/ai-pipeline/prompts/` :

- `extract-v3-v1.0.md` — Extraction V3 (vision GPT-4o)
- `read-releve-v1.0.md` — Lecture relevé de mesures
- `merge-validate-v1.0.md` — Fusion + validation croisée
- `match-catalogue-v1.0.md` — Matching catalogue fournisseur
- `plan-elec-v1.0.md` — Génération plan élec NFC 15-100
- `plan-plomb-v1.0.md` — Génération plan plomberie
- `fiche-pose-v1.0.md` — Génération fiche de pose

NOTE : Chaque prompt commence par un préfixe standardisé : *version, date, auteur, changelog*. Les modifications de prompts passent par review code obligatoire (impact production).

C. Catalogues fournisseurs (sources)

Marque	Catalogue	Format source	Fréquence maj
Blum	Complet 8k refs	CSV public via API partenaire	Hebdomadaire
Hettich	Complet 12k refs	PDF scrapé mensuellement	Mensuelle
Häfele	Top 3k refs	Catalogue public + scraping	Trimestrielle
Appareils (Bosch, Neff, AEG, Siemens)	56 appareils cuisine	Fiches techniques constructeur	Semestrielle

D. Checklist de validation avant merge PR

- Tests unitaires passent (Vitest) + couverture $\geq 90\%$
- Tests integration passent (supertest)
- Lint + typecheck pass (ESLint + tsc --noEmit)
- Pas de secret hardcodé (gitleaks scan)
- Documentation à jour (README + JSDoc exports publics)
- Prisma migration testée sur DB de staging
- Golden dataset re-run (non-régression IA)
- Mode ghost testé si modification de prompt
- Load test si modification worker (100 jobs concurrents)
- Review par au moins 1 dev senior

E. Commandes de dev utiles

```
# Setup dev
pnpm install
pnpm --filter @avra/api exec prisma migrate dev
pnpm --filter @avra/api exec prisma generate

# Run dev
pnpm --filter @avra/api dev          # API NestJS
pnpm --filter @avra/web dev          # Frontend Next
pnpm --filter @avra/api dev:worker  # Worker Bull standalone

# Tests
pnpm --filter @avra/api test          # tous les tests unit/integ
pnpm --filter @avra/api test:golden  # non-régression IA
pnpm --filter @avra/api test:ghost   # mode ghost vs prod
pnpm --filter @avra/web test:e2e     # Playwright

# Build prod
pnpm --filter @avra/api build
pnpm --filter @avra/web build

# DB
pnpm --filter @avra/api exec prisma studio # UI DB browser
pnpm --filter @avra/api exec prisma migrate deploy # appliquer migs prod

# Debug
pnpm --filter @avra/api dev:worker -- --inspect # debug worker
tail -f logs/plan-technique.log | jq .          # logs JSON structurés
```

F. Environnement variables requises

```
# IA providers
OPENAI_API_KEY=sk-proj-...
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=AI...

# Queue
REDIS_URL=redis://...
REDIS_TLS=true

# Storage
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
SUPABASE_PLANS_BUCKET=plans-techniques

# Feature flags
FEATURE_PLAN_TECHNIQUE_ENABLED=true
FEATURE_PLAN_TECHNIQUE_GHOST_MODE=false

# Limits
PLAN_TECHNIQUE_MAX_CONCURRENT_JOBS=10
PLAN_TECHNIQUE_IA_TIMEOUT_MS=300000
PLAN_TECHNIQUE_MAX_RETRY=2

# Monitoring
SENTRY_DSN=https://...@sentry.io/...
GRAFANA_CLOUD_API_KEY=glc...
```

Note finale pour Claude Code

Ce cahier des charges est conçu pour être donné **tel quel** à Claude Code ou à un dev senior. Les 17 sections + 6 annexes couvrent l'intégralité du périmètre.

Ordre recommandé d'implémentation

- 1. Lire intégralement ce document avant d'écrire une ligne de code
- 2. Démarrer par la section 5 (schéma Prisma) — c'est la fondation
- 3. Ensuite section 6 (endpoints stubs) pour avoir un skeleton
- 4. Puis section 7 (pipeline IA) + section 8 (règles métier) **en parallèle**
- 5. Section 9 (PDF) en dernier — dépend des deux précédentes
- 6. Frontend (section 10) à faire **après** un premier MVP backend bout en bout
- 7. Tests (section 11) à écrire **au fur et à mesure**, pas à la fin

Principes directeurs

- **Pas de compromis sur la qualité** — senior+++ validé par le PO
- **Tests d'abord** pour les règles métier (TDD strict en section 8)
- **Audit exhaustif** sur chaque appel IA (logs + coût + latence)
- **Pas de shortcut sur la sécurité** — ownership, quotas, rate limiting à chaque endpoint
- **Documentation continue** — chaque PR met à jour le README du module

Document généré le 24/04/2026 à 23:11

Version 1.0 — Prêt pour implémentation

Auteur : Esteve Boucheret (lumeasolutions@outlook.fr)

© 2026 AVRA — Document confidentiel — Ne pas diffuser